

Lab 0 : Gitlab 实验报告

25300180078 沈越

数学科学学院

实验任务

1. (1) 暂无多人协同开发的经历。

(2) 单步提交具有粗粒度的特点，改了什么就得全提交。“暂存-提交”两个步骤可以使粒度更细，可按块/行精准选择提交；单步提交一旦改动立即进入历史，容错率低。“暂存-提交”两个步骤提交前可随时调整暂存内容，容错率高；单步提交依赖本地临时标记，“暂存-提交”两个步骤依赖暂存区状态管理，清晰受控；单步提交的提交历史容易混乱，暂存-提交”两个步骤可使提交历史清晰可读，便于追溯。

(3) 二者作用范围不同。Git branch 仅支持显示本地分支，git branch -a (-a 相当于 -all) 既支持本地分支也支持远程追踪分支。

2. 实验步骤：

(1) 按照“作业提交流程：使用模板仓库”中的描述，创建自己的仓库（命名为“TestLab”）。

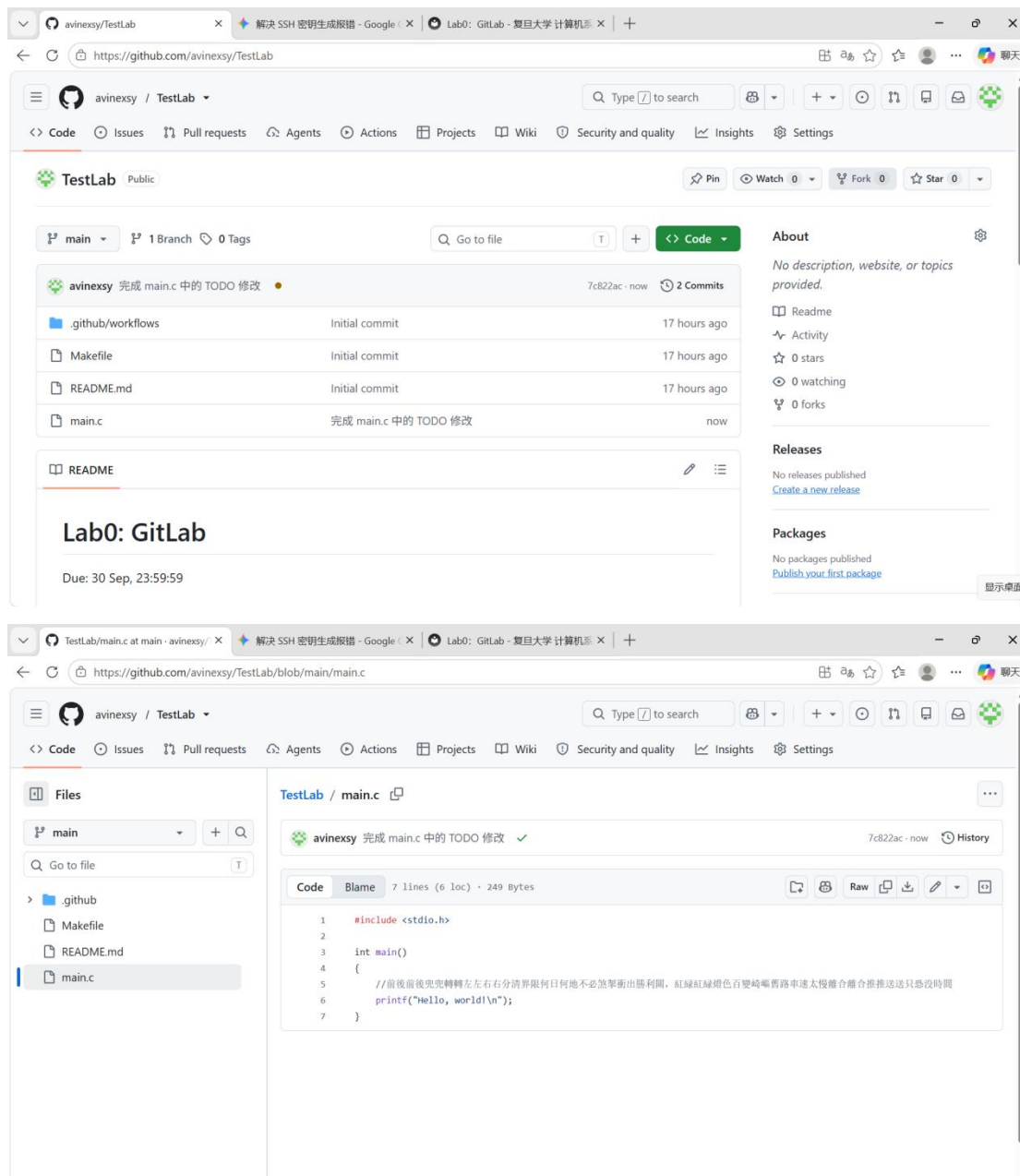
(2) 打开 Git Bash, 使用 git clone 克隆仓库到本地, 使用 cd TestLab 进入刚刚生成的文件夹 TestLab。

(3) 在文件夹中找到 main.c 文件，在 VSCode 打开该文件并修改代码。

(4) 使用 git add main.c 将修改后的文件添加到暂存区，使用 git commit -m “完成 main.c 中的 TODO 修改” 提交修改并加上说明。

(5) 使用 git push 将修改、提交的记录同步推送到远程仓库。

实验结果



3. 文章概括：

“Commit Message 规范”：Git 代码提交时必须有 commit message。Commit message 具有方便快捷浏览、便于快速查找信息、可直接生成 Change log 等优势。

Commit message 包括 Header（必需）、Body、Footer 三部分。如果当前 commit 用于撤销以前的 commit，则必须以 revert: 开头，且 Body 部分有统一的格式。安装 Commitizen 后，可使用 git cz 生成符合格式的 commit message。可使用 validate-commit-msg 检查 commit message 的格式规范。若所有 commit message 符合 Angular 格式，则可自动生成 Change log。

“Gitflow 使用规范”：Git 操作是优秀的分支管理策略，可解决编程中的代码冲突和版本问题。Gitflow 具有 master、develop、feature、release、hotfix 等关键分支，它们分别用于代码的开发、测试、排查、上线、客户反馈等不同阶段。

“为什么要学习 Git”：（1）现如今 AI 研发具有高度迭代性和实验性的特点，在模型优化的过程中，很可能需要频繁修改代码。使用 Git 可以清晰记录修改历史，便于追踪。遇到新的改进思路，还可以新建分支独立开发。（2）目前 AI 的开源氛围浓厚，使用 Git 可以熟练地 clone 前沿论文等开源项目，获取优质资源。（3）在高校的课题组项目、AI 竞赛或企业实习中，算法研发多为多人并行开发，Git 允许前端（如 Web 可视化）、后端（数据接口）和算法工程师（模型训练）在各自的分支上同步推进，最后通过合并集成，可显著提高效率。（4）Git 是迈向 AI 工程化的必备技能；熟练使用 Git 是成为具备工程落地能力的复合型人才的基础。

4. 实验步骤：

（1）使用 cd TestLab 进入 TestLab。使用 git switch -c feature 创建新的 feature 分支并直接切换过去。

```
25300@shenyue MINGW64 ~/Desktop
$ cd TestLab

25300@shenyue MINGW64 ~/Desktop/TestLab (main)
$ git switch -c feature
Switched to a new branch 'feature'
```

(2) 打开 main.c 文件，修改其中一行代码，保存文件后使用 git add main.c 以及 git commit-m “在 feature 分支修改 main.c” 在 Git Bash 提交。

```
25300@shenyue MINGW64 ~/Desktop/TestLab (feature)
$ git add main.c

25300@shenyue MINGW64 ~/Desktop/TestLab (feature)
$ git commit -m "在 feature 分支修改 main.c"
[feature b73934e] 在 feature 分支修改 main.c
1 file changed, 1 insertion(+), 1 deletion(-)
```

(3) 使用 git switch main 切换回 main 分支。

```
25300@shenyue MINGW64 ~/Desktop/TestLab (feature)
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

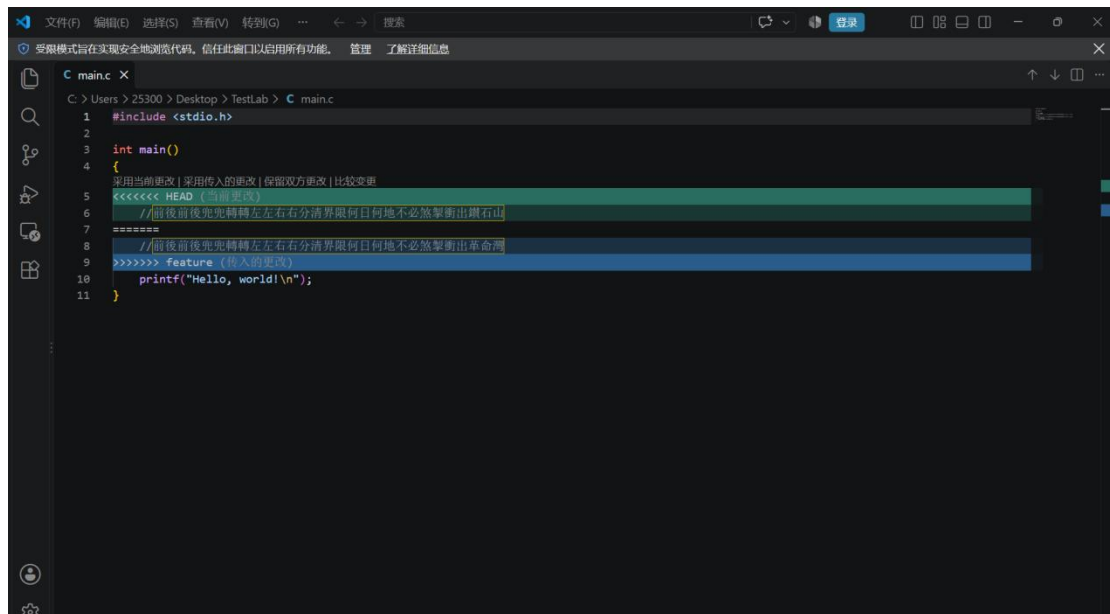
(4) 再次打开 main.c，修改一行代码（两次修改的是同一行，但内容不同），类似地使用 add 和 commit 提交。

```
25300@shenyue MINGW64 ~/Desktop/TestLab (main)
$ git add main.c

25300@shenyue MINGW64 ~/Desktop/TestLab (main)
$ git commit -m "在 main 分支修改 main.c"
[main 9812412] 在 main 分支修改 main.c
1 file changed, 1 insertion(+), 1 deletion(-)
```

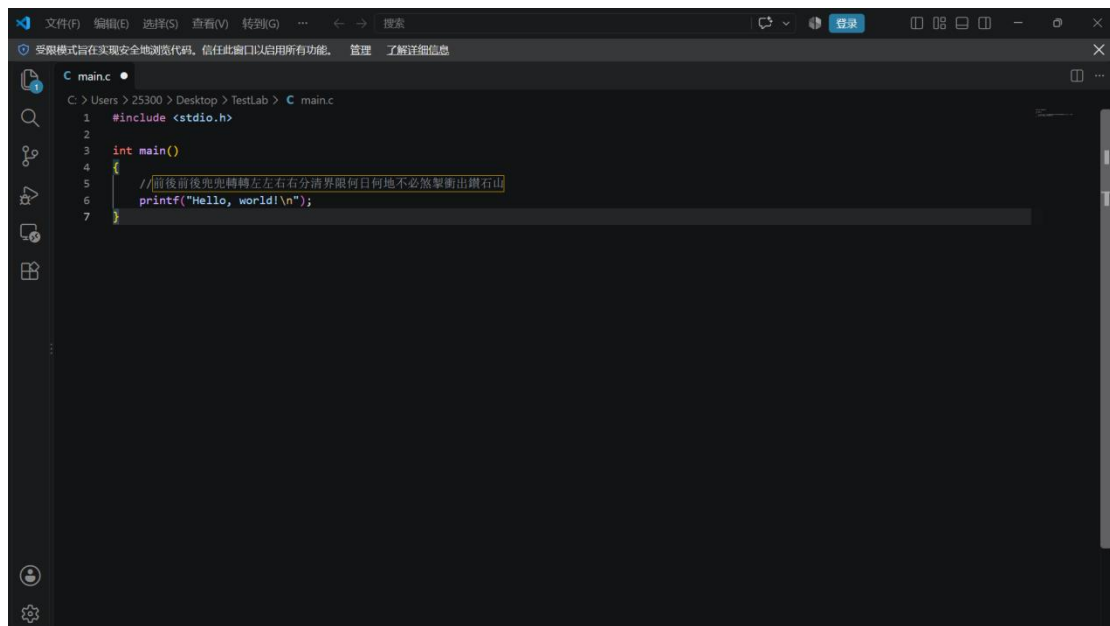
(5) 使用 git merge feature 将两个分支合并，出现如图所示的冲突。

```
25300@shenyue MINGW64 ~/Desktop/TestLab (main)
$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```



```
1 #include <stdio.h>
2
3 int main()
4 {
5     //采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6     <<<<<< HEAD (当前更改)
7     //前後前後兜轉轉左左右右分清界限何日何地不必煞掣衝出鑽石山
8     =====
9     //前後前後兜轉轉左左右右分清界限何日何地不必煞掣衝出革命灣
10    >>>>>> feature (传入的更改)
11    printf("Hello, world!\n");
12 }
```

(6) 在 VSCode 中手动修改代码。这里删除所有冲突标记，并删除“革命湾”句，保留“钻石山”句。



```
1 #include <stdio.h>
2
3 int main()
4 {
5     //前後前後兜轉轉左左右右分清界限何日何地不必煞掣衝出鑽石山
6     printf("Hello, world!\n");
7 }
```

(7) 重新执行 add 和 commit，使用 push 推送至远程仓库，即可解决冲突。

